Ejercicio práctico 1.2

«La biblioteca del barrio»

Nombre	Diego Alexander Cabrera Carrion
Curso	Tercer Ciclo “A”
Fecha	12/10/2026

Asignatura	Herramientas informáticas para el despliegue de diagramas
Unidad	1. Principios de la arquitectura web en el diseño y desarrollo de aplicaciones backend
Tema	1.2 Métodos HTTP y operaciones sobre recursos
Docente	Jhostin Omar Vacacela Saca
Duración	60 minutos, dentro de las 4 horas de clase
Modalidad	En parejas

Objetivo

Elegir el método HTTP adecuado para cada operación sobre un recurso, implementarlo en Nest.js y analizar la respuesta del servidor.

Situación

La biblioteca de un barrio quiere pasar su catálogo de fichas de papel a un servicio web. Cada libro tiene un id, un título, un autor y un campo disponible que vale true cuando el libro está en el estante.

Tu pareja y tú construyen el servicio que expone ese catálogo. El recurso se llama libros y vive en la dirección /libros.

Instrucciones generales

1. Trabaja con tu pareja. Las partes A a C se resuelven en este mismo documento; la parte D se resuelve en código.
2. Entrega un archivo .zip con este documento respondido, la carpeta del proyecto sin node_modules, la colección de Postman exportada en .json y la captura de las pruebas en verde.
3. Para ejecutar el proyecto usa Node.js 22 o superior: npm install y luego npm start.
4. Si usas inteligencia artificial, decláralo en la parte E. Sin declaración, la parte E no puntúa.

Parte A. Leer y predecir (20 puntos)

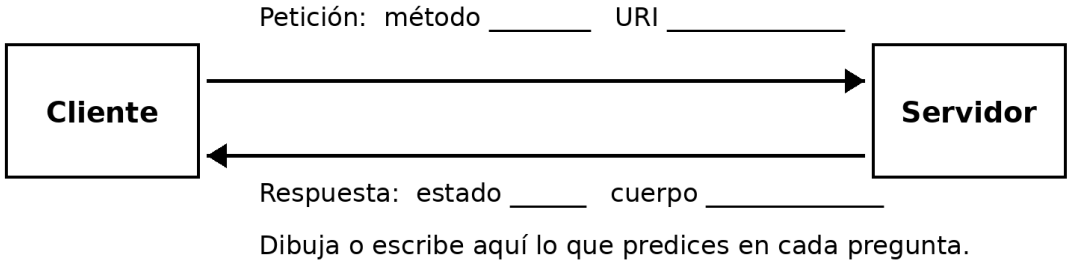
El servicio recién iniciado tiene estos tres libros y la variable siguienteId vale 4:

```
{ id: 1, titulo: "Cien años de soledad", autor: "García Márquez", disponible: true }
{ id: 2, titulo: "El túnel",             autor: "Sabato",          disponible: true }
{ id: 3, titulo: "Rayuela",              autor: "Cortázar",        disponible: false }
```

Y este es el código de dos de sus endpoints:

```
@Get(':id')
obtener(@Param('id', ParseIntPipe) id: number) {
  return this.libros[this.indiceDe(id)];
  // si no existe, indiceDe lanza NotFoundException(`No existe el libro ${id}`)
}

@Post()
crear(@Body() cuerpo, @Res({ passthrough: true }) res) {
  const nuevo = { id: this.siguienteId++, ...cuerpo };
  this.libros.push(nuevo);
  res.setHeader('Location', `/libros/${nuevo.id}`);
  return nuevo;
}
```



Dibuja o escribe aquí lo que predices en cada pregunta.

Las peticiones se envían en este orden, una tras otra, sobre el servicio recién iniciado. Escribe el estado y el cuerpo que devolverá cada una.

A1 · 5 puntos
GET /libros/2
Tu respuesta
Estado: 200 OK. Cuerpo: { "id": 2, "titulo": "El túnel", "autor": "Sabato", "disponible": true }

A2 · 5 puntos

GET /libros/9

Tu respuesta

Estado: 404 Not Found

Cuerpo: { "message": "No existe el libro 9", "error": "Not Found", "statusCode": 404

A3 · 5 puntos

POST /libros con el cuerpo {"titulo":"Pedro Páramo","autor":"Rulfo","disponible":true}

Tu respuesta

Estado:201 Created

Cabecera Location: /libros/4

Cuerpo: { "id": 4, "titulo": "Pedro Páramo", "autor": "Rulfo", "disponible": true }

A4 · 5 puntos

La misma petición A3, enviada otra vez. ¿Qué estado y qué Location devuelve? ¿Cuántos libros hay en el catálogo después de A4?

Tu respuesta

Estado: 201 Created

Cabecera Location: /libros/5, porque cada petición POST crea un nuevo registro.

Libros en el catálogo: 5

Parte B. Completar (20 puntos)

Completa los cinco espacios en blanco del controlador. Cada espacio vale 4 puntos. Los comentarios dicen qué debe hacer cada función.

```
@Controller('libros')
export class LibrosController {
  @(1)______()                          // lista la colección
  listar(@Query('autor') autor?: string) { /* ... */ }

  @(2)______(':id')                     // reemplaza el libro completo
  reemplazar(@Param('id', ParseIntPipe) id: number, @Body() cuerpo) { /* ... */ }

  @(3)______(':id')                     // cambia solo los campos enviados
  modificar(@Param('id', ParseIntPipe) id: number, @Body() cambios) { /* ... */ }

  @(4)______(':id')                     // elimina el libro
  @HttpCode(HttpStatus.(5)______)       // responde sin cuerpo
  eliminar(@Param('id', ParseIntPipe) id: number) { /* ... */ }
}
```

Espacio	Tu respuesta
(1)	Get
(2)	Put
(3)	Patch
(4)	Delete
(5)	NO_CONTENT

Parte C. Detectar y corregir errores (20 puntos)

Cada situación tiene un error en la elección del método o de la dirección. Explica cuál es y escribe cómo lo corregirías. Cada una vale 5 puntos.

C1 · 5 puntos
Un compañero escribió: @Get(':id/borrar') borrar(...) { /* elimina el libro */ }
Tu respuesta
Error: Usar GET /libros/:id/borrar para eliminar. Las URLs deben identificar recursos, no acciones. Corrección: @Delete(':id') eliminar(...)— usar DELETE /libros/:id

C2 · 5 puntos
Otro escribió: @Put(':id') marcarPrestado(@Param('id') id, @Body() cuerpo: { disponible: boolean }) { /* guarda solo el campo disponible */ }
Tu respuesta
Error: Usar PUT para cambiar únicamente el cambio disponible. PUT reemplaza el recurso completo, se perderían título y autor. Corrección: Usar @Patch(':id'), porque permite modificar solo el campo necesario sin reemplazar todo el recurso.

C3 · 5 puntos
Un tercero escribió, dentro de @Controller('libros'): @Post('crearLibro') crear(...) { /* crea un libro */ }
Tu respuesta
Error: Usar POST / libros/crearLibro. El verbo no deber estar en la ruta. Corrección: @Post() sin segmentación adicional, porque POST sobre /libros ya expresa “crear”.

C4 · 5 puntos
Alguien probó PUT /libros/1 con el cuerpo {"disponible":false} para marcar el libro como prestado y recibió 200 con este cuerpo: {"id":1,"disponible":false}. ¿Qué pasó con el título y el autor? ¿Qué petición debió enviar?
Tu respuesta
Qué pasó: Como PUT reemplaza el recurso completo, el libro quedó solo con id y disponible : se perdieron el título y el autor. Petición correcta: PATCH /libros/1 con {"disponible": false} , porque modifica solamente los campos enviados.

Parte D. Escribir desde cero (30 puntos)

Implementa el servicio de la biblioteca. Usa como modelo el proyecto api-productos visto en clase.

1. Copia api-productos y renómbralo libros-api. Cambia el recurso a libros con los campos id, titulo, autor y disponible, y carga los tres libros de la parte A.

2. Crea los seis endpoints de la tabla. Si el id no existe, el servicio debe responder 404.
3. En Postman crea una colección «Biblioteca» con una petición por endpoint. En cada una escribe una prueba que compruebe el estado esperado.
4. Ejecuta la colección completa y guarda una captura con todas las pruebas en verde.

Operación	Método y dirección	Estado esperado	Puntos
Listar, con filtro opcional ?autor=	`GET /libros`	200	5
Obtener un libro	`GET /libros/{id}`	200, o 404 si no existe	5
Crear un libro (envía Location)	`POST /libros`	201	5
Reemplazar un libro completo	`PUT /libros/{id}`	200	5
Cambiar campos de un libro	`PATCH /libros/{id}`	200	5
Eliminar un libro	`DELETE /libros/{id}`	204	5

Criterio de éxito: las seis pruebas de la colección pasan en verde y puedes explicar, para cada petición, por qué elegiste ese método.

Parte E. Reflexión y declaración de uso de IA (10 puntos)

E1 · 6 puntos

Un cliente móvil pierde la señal justo después de enviar POST /libros y reintenta solo. ¿Qué puede pasar en el catálogo? ¿Podría reintentar sin riesgo un DELETE /libros/3? Apóyate en lo que viste en las partes A y D.

Tu respuesta

Existe el riesgo directo de duplicar registros en el catálogo. Como el método POST no es idempotente, cada solicitud recibida es tratada de forma independiente, asignando un nuevo identificador mediante siguienteId. Si la primera petición se completó en el servidor y lo único que falló fue la entrega de la respuesta al cliente, el reintento creará un segundo libro idéntico con otro ID. Esto coincide con lo observado en los ejercicios A3 y A4, donde enviar dos veces la misma información generó los IDs 4 y 5, incrementando el catálogo a 5 elementos.

Sí, se puede reintentar porque DELETE es una operación idempotente. La primera llamada elimina el recurso devolviendo un código 204, mientras que la segunda fallará con un 404 al no encontrar el recurso (tal como se manejó en la parte D). El estado final del catálogo no cambia: el libro 3 ya no existe. Además, dado que siguienteId incrementa de forma secuencial y no recicla identificadores previos, no existe peligro de eliminar por accidente un registro nuevo.

Declaración de uso de inteligencia artificial · 4 puntos
Marca una opción: [ ] No usé IA. [X] Usé IA. Herramienta usada: GEMINI Para qué la usé: Para guiarme en algunos puntos que no entendia o que no lograba solucionar el error. Qué revisé y corregí yo: Verifiqué el funcionamiento de cada endpoint en Postman y confirmé que todas las pruebas pasaran en verde.

Se permite el uso de IA con declaración de la herramienta y el alcance. Como referencia, el reporte de Turnitin no debe pasar de 30 % de contenido posiblemente generado por IA ni de 10 % de similitud.

Evaluación

Instrumento del PEA para este tema: trabajo práctico de codificación. Se califica con la siguiente lista sobre 100 puntos.

Parte	Qué se evalúa	Puntos
A	Predice el estado, las cabeceras y el cuerpo de cada petición (4 × 5)	20
B	Elige el decorador y el estado correctos (5 × 4)	20
C	Detecta el error y propone el método o la dirección adecuados (4 × 5)	20
D	Cada endpoint funciona y su prueba de Postman pasa en verde (6 × 5)	30
E	Reflexión (6) y declaración de uso de IA (4)	10
Total		100

Lista de cotejo de entrega

1. El .zip incluye este documento, el proyecto sin node_modules, la colección de Postman y la captura.
2. El proyecto arranca con npm install y npm start sin pasos extra.
3. La colección tiene seis peticiones y todas sus pruebas pasan en verde.
4. La declaración de uso de IA está completa.

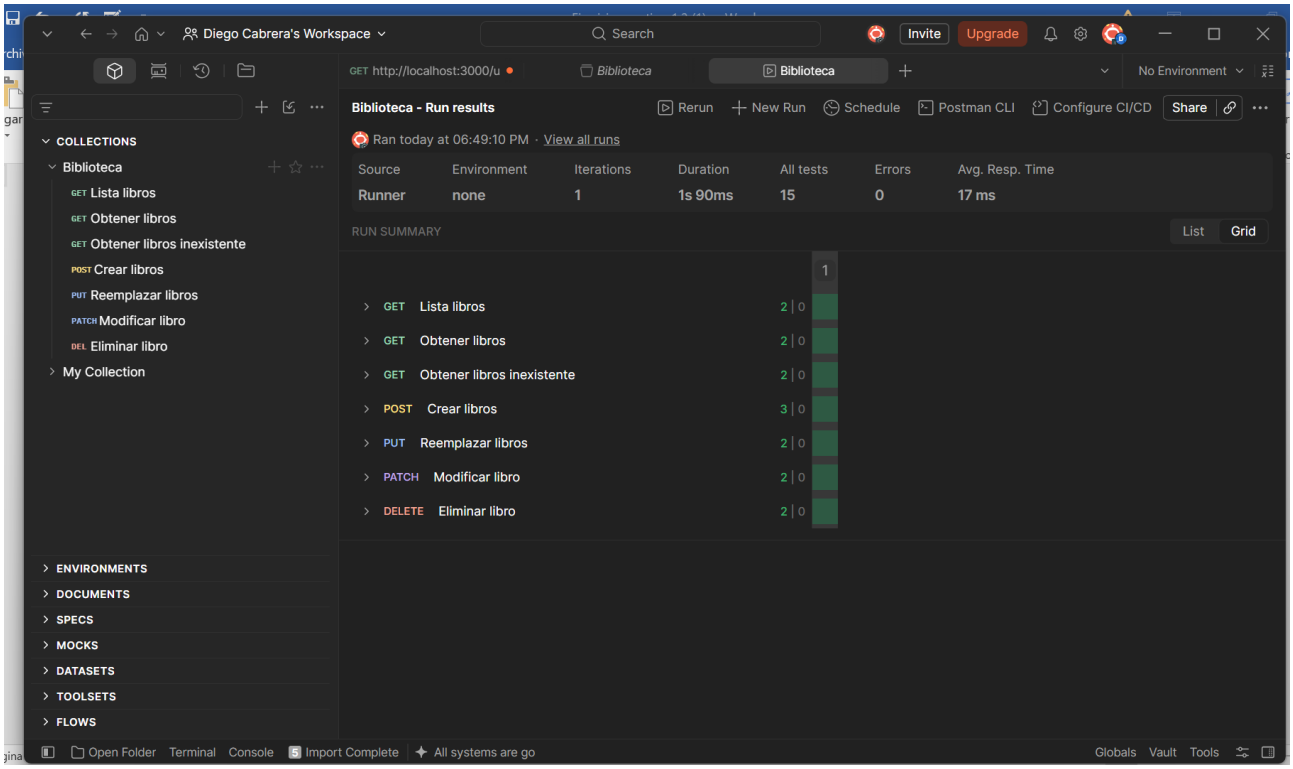
Referencias

Mozilla Developer Network. (s. f.). HTTP request methods. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). HTTP semantics (RFC 9110). IETF. https://www.rfc-editor.org/rfc/rfc9110

NestJS. (s. f.). Controllers. https://docs.nestjs.com/controllers

EVIDENCIAS



Registro Detallado del Proceso de Investigación y Desarrollo

Documentación del flujo de consultas e interacciones en el desarrollo (libros-api).

#	Emisor	Contenido / Consulta	Objetivo / Información Clave	Conexión con el siguiente paso
1	Usuario	"Oye, mi GET por ID no encuentra el libro. Le mando el 1 en Postman y me dice que no existe, pero sí está en la lista."	**Objetivo:** Buscar un libro por su ID al hacer el endpoint.	Lleva a la respuesta del Asistente identificando el problema de tipos.
2	Asistente	Explica que el ID en la URL llega como string y sugiere agregar ParseIntPipe en el @Param para convertirlo a número.	**Información:** Manejo de tipos nativo y pipes de NestJS.**Solución:** ParseIntPipe.	Permite al usuario corregir el endpoint y pasar a probar las actualizaciones.
3	Usuario	"Estaba probando actualizar solo el estado de 'disponible'	**Objetivo:** Hacer actualización parcial sin	Lleva al Asistente a

#	Emisor	Contenido / Consulta	Objetivo / Información Clave	Conexión con el siguiente paso
		a false, pero al hacerlo se me borró el título y el autor del libro. ¿Por qué?"	perder o dañar datos existentes.	explicar la diferencia entre los métodos HTTP.
4	Asistente	Aclara que PUT reemplaza todo el objeto y recomienda cambiar el método a PATCH, que modifica solo lo enviado.	**Información:** Diferencia entre PUT y PATCH.**Solución:** Usar decorador @Patch().	Ayuda al usuario a aplicar el método correcto y revisar el diseño de sus rutas.
5	Usuario	"Una duda, ¿está bien poner rutas como /:id/borrar o /crearLibro en el controlador de NestJS?"	**Objetivo:** Identificar y corregir errores comunes en la nomenclatura de los endpoints.	Lleva a la respuesta del Asistente sobre estándares de diseño REST.
6	Asistente	Indica que las URLs deben identificar recursos y no acciones, sugiriendo usar decoradores limpios como @Delete(':id') o @Post().	**Información:** Diseño de APIs REST.**Solución:** Eliminar verbos de las rutas.	Permite dejar los controladores con estándar REST antes de probar qué pasa con los fallos.
7	Usuario	"Oye, ¿qué pasa si se me corta el internet al mandar un POST y mi app lo reintenta sola? ¿Y si me pasa con un DELETE?"	**Objetivo:** Analizar el comportamiento del servidor ante fallos de red y reintentos.	Lleva al Asistente a explicar el concepto de idempotencia en peticiones.
8	Asistente	Explica que POST no es idempotente (duplicará registros) pero DELETE sí lo es (la segunda vez dará 404 sin riesgo de dañar nada).	**Información:** Idempotencia.**Claves:** Riesgo de duplicidad en POST.	Da la base teórica final para poder armar la colección de pruebas en Postman.
9	Usuario	"¿Cómo hago para verificar en la pestaña de tests de Postman si está correcto? Vi que mi compañero tiene validaciones extra."	**Objetivo:** Dejar lista la colección con pruebas automáticas para comprobar los estados.	Lleva al Asistente a mostrar snippets de código nativos para Postman.

#	Emisor	Contenido / Consulta	Objetivo / Información Clave	Conexión con el siguiente paso
10	Asistente	Enseña a usar el código pm.response.to.have.status() en cada petición para verificar automáticamente si devuelve 200, 201 o 404.	**Información:** Automatización de pruebas.**Solución:** Snippet pm.response.to.have.status().	Permite correr el Runner completo y obtener las 6 pruebas en verde para la entrega final.